Technical write-up — GenAI Report Agent

1. Problem understanding

The task is to build an agent that, every hour:

1. collects recent articles on set topics (AI Regulation, Technology Updates, UK Economy) from trusted sites (BBC News, GOV.UK),
2. turns them into a short report: a 100–150 word summary, 3–5 key takeaways, and the organisations and terms mentioned,
3. lets a user chat about the latest reports, including vague questions like *"What's happening nowadays?"*

The normal case is easy. The hard part is the edge cases: hours with no news, the same story appearing twice, websites failing to load, the model writing too many words or inventing a number, follow-up questions like "why?" that make no sense on their own, and questions the reports don't cover.

2. Solution overview

The system has two LangGraph workflows that share a simple file store.

Report workflow (`report_graph.py`, run every hour by APScheduler):

Step	What it does
Plan	Decides which sources to fetch and which URLs were already reported.
Collect	Runs the scraping tool through LangGraph's `ToolNode`: read the RSS/Atom feed, keep relevant articles, download each page with BeautifulSoup, and clean the text.
Process	Removes duplicates and ranks articles by relevance. If there's nothing new, it stops and keeps the old reports.
Summarise	Each article is shortened to a few factual notes, then the notes are combined into one report with a fixed structure.
Evaluate	Checks the report (see section 4).
Revise	If a check fails, the report goes back to the model with the exact problem, at most twice.
Save	Writes the report as JSON and Markdown and remembers which articles were used.

You can also run it by hand with `python main.py report`.

Chat workflow (`chat_graph.py`): analyse → retrieve → respond, with memory of the conversation.

- **Analyse** works out what kind of question it is (a vague overview, a specific topic, a follow-up, or small talk) and rewrites follow-ups into full questions.
- **Retrieve** fetches the latest report per topic for vague questions, or searches the reports for specific ones.
- **Respond** answers using only the reports, cites them as [R1], [R2], and says when something isn't covered.

3. Key GenAI concepts used and why

- **Map-reduce summarisation.** Summarising each article first keeps each model call small, and the final step can compare stories instead of just joining summaries together.

- **Structured output.** The model fills a fixed template (summary, takeaways, entities), so there is no messy text to parse and the result is easy to check.
- **Evaluate and revise loop.** Models often miss word limits. Giving them specific feedback ("172 words, must be 100–150") fixes this more reliably than a longer prompt.
- **Grounding.** The chat answers only from the reports, and every number and organisation in a report must appear in the source articles. This prevents confident but wrong answers.
- **Context engineering for chat.** The model sees short, dated, labelled reports rather than whole web pages, plus only the last few messages. That makes answers cheaper, more focused, and easy to cite.
- **Handling vague questions.** "What's happening nowadays?" has no useful search words, so it's sent straight to a summary of the latest report for each topic.
- **Saying "I don't know".** If a search finds no good match, the agent says the topic isn't covered instead of guessing.

4. Logging and evaluation

- **Logs:** every step writes a line to `logs/agent.jsonl` with how long it took, how many articles went in and out, any errors, and the evaluation results.
- **Report checks (every run):** word count, number of takeaways, whether numbers and organisations appear in the sources, how many articles are covered, and an LLM judge score for accuracy.
- **Chat checks (`python main.py eval`):** test questions covering a vague overview, a specific fact, a follow-up, an off-topic question and small talk.
- **Results from live runs (`claude-sonnet-5-5`):** 8 reports, all passed, average score 0.97, summaries of 130–133 words, and no revisions needed.
- **Tests:** 11 automated tests that use a fake LLM, so they run without an API key.

5. Alternatives considered and final choices

Choice	Alternatives	Why
RSS feeds to find articles, then scrape each page	Scrape whole news pages; Newspaper3k	Feeds are stable and list the newest stories. Newspaper3k is no longer maintained.
Fixed list of sources	Let the LLM decide what to fetch	The sources never change, so an LLM would only add cost and unpredictability.
Map-reduce summarising	Send everything in one prompt	One big prompt breaks when there are many or long articles.
Rule checks plus an LLM judge	LLM judge only; ROUGE	There are no reference summaries for live news, so ROUGE can't be used. Rule checks are free and exact.
BM25 keyword search	Embeddings and a vector database	Only a few dozen short reports, so keyword search is fast, simple and easy to explain.
JSON and Markdown files	A database	No setup and easy to inspect.
APScheduler	cron, Airflow	Runs inside the app and prevents runs from overlapping. Airflow is too much for one job.
Offline fallback mode	Require an API key	Lets anyone test the project without a key.

6. Limitations and next steps

- **Issues found in live testing:** the model rejected a custom `temperature` setting, so it's no longer sent. It also sometimes left out the entities field. The agent now fills that field with a fallback extractor instead of discarding the whole report.
- Website layouts change, so the scraping code will need occasional updates.

- The same story can appear in two topics (for example AI Regulation and Technology Updates).
- Next steps: add embedding search alongside keyword search, stream chat answers, and keep chat memory after a restart.